

DV Framework — Coder Brief (v1)

2026-09-19 · @Someone

1. Goal and v1 scope

Build DV: a stateless, server-driven full-stack framework where one `.dv` file declares domain, database access and UI, and the runtime serves a working SPA with no manual frontend build. It replaces Vaadin Flow's model: no UI tree in server memory, no sticky sessions, no separate Vite terminal.

v1 is done when a developer writes `User.dv` and `UserLogin.dv`, runs one command, and gets a working login plus User CRUD app, rendered by **either** the React or the Lit target from the same source, with live diagnostics in IntelliJ.

In scope for v1

- DV language: `@Domain`, `@Database`, `@Component`, `@JS` blocks
- Compiler core shared by the LSP server and the runtime
- Two UI targets behind one interface: React and Lit
- LSP server usable from IntelliJ via LSP4IJ
- Vert.x 5 BFF: component serving, signed stateless actions, Redis sessions
- esbuild vendor bundling driven by the runtime, npm auto-install
- Dev loop: file watch, recompile, browser reload, error overlay

Out of scope for v1 (do not build, do not stub)

- GraalPy / Python blocks
- esbuild as WASM inside the JVM
- JSON layout AST for mobile renderers
- SSR, Datastar, HTML-over-the-wire
- Virtual grids, multi-step wizard state, server push
- Language intelligence inside `@JS` blocks
- A native JetBrains plugin

2. Architecture and modules

DV is a compiler, not an interpreter: `.dv` source compiles to ES modules, SQL, validators and a route table. Code only executes at runtime in three places: action handlers, custom validators and `@JS` blocks.

```

flowchart LR
  SRC[".dv files"] --> CORE["dv-compiler<br/>parse, check, model"]
  CORE --> LSP["dv-lsp<br/>IntelliJ via LSP4IJ"]
  CORE --> IR["UI IR"]
  IR --> REACT["dv-emitter-react"]
  IR --> LIT["dv-emitter-lit"]
  CORE --> RT["dv-runtime<br/>Vert.x 5 BFF"]
  REACT --> RT
  LIT --> RT
  RT --> BROWSER["Browser SPA"]

```

One compiler core feeds the editor, both emitters and the runtime, so an error reads the same everywhere.

Module	Contents	May depend on
<code>dv-compiler</code>	ANTLR4 grammar, AST, symbol table, semantic checks, diagnostics, UI IR, <code>Emitter</code> interface	ANTLR runtime only
<code>dv-emitter-react</code>	React emitter, React component maps	<code>dv-compiler</code>
<code>dv-emitter-lit</code>	Lit emitter, Lit component maps	<code>dv-compiler</code>
<code>dv-lsp</code>	LSP4J server over stdio	<code>dv-compiler</code> , LSP4J
<code>dv-runtime</code>	Vert.x BFF, esbuild service, dev loop, polyglot host	all of the above, Vert.x, GraalVM polyglot
<code>dv-cli</code>	Single fat jar: <code>dv dev</code> , <code>dv lsp</code> , <code>dv build</code>	all
<code>dv-example</code>	<code>User.dv</code> , <code>UserLogin.dv</code> , <code>package.json</code>	none (consumed by tests)

`dv-compiler` must never import Vert.x, LSP4J or GraalVM. That rule is what lets the LSP start in under a second.

Stack: Java 25, Maven multi-module, Vert.x 5 (futures API only), ANTLR 4.13.x, LSP4J, GraalVM polyglot + GraalJS matching the JDK in use, PostgreSQL through the Vert.x reactive PG client, Redis through the Vert.x Redis client, esbuild from npm. Pin every version in the parent POM; take current stable releases rather than the versions in the source transcript.

3. DV language and compiler core

The syntax below is the v1 reference; it normalises the founder's sketch (braces, consistent annotation arguments). `String` is the default field type.

```
@Domain
var User {
  @id(uuid)                                userId
  @validator(notEmpty, size(min: 8), unique) userName
  @validator(notEmpty, size(min: 2))       firstName
  @secret                                  password
}

@Database(User, "users") {
  crud
  query login(userName, password): verify
  query existsByUserName(userName): exists
}

@Component
var UserLogin {
  ui: FormLogin
  title: "Please login"
  fields {
    userName
    password (passwordField)
  }
  onSubmit: User.login
}

@JS {
  function strongPassword(pwd) { return pwd.length >= 8 && /[A-Z]/.test(pwd);
}
}
```

Grammar

- ANTLR4, one grammar file, visitor mode. Keep ANTLR's default error recovery on.
- `@JS { ... }` is lexed as one opaque token with balanced-brace matching; the body is not parsed in v1.
- Optional per-component override blocks: `@React { ... }` and `@Lit { ... }` hold a hand-written template for that target and replace the generated one.

Compiler core requirements (these exist because the LSP consumes the same code)

1. Parsing never throws. A half-typed file returns a partial AST plus diagnostics.

2. Every AST node carries a source range: file URI, start and end line and column.
3. AST is a sealed interface hierarchy of records. No mutable nodes.
4. Semantic analysis returns `List<Diagnostic>` with severity, code, message and range. It never throws for user errors.
5. A workspace-level symbol table resolves references across files: `@Database(User, ...)` and `onSubmit: User.login` point at declarations elsewhere.
6. Public entry point: `Workspace.update(uri, text)` returns a `CompilationResult` holding model, diagnostics and symbols. Whole-file reparse per change is fine; files are small.
7. No Truffle language implementation in v1. DV is declarative; nothing in it needs a Truffle interpreter.

Semantic checks for v1: unknown annotation, unknown validator, bad validator argument, duplicate declaration, unresolved domain or query reference, field in a component not present in the bound domain, `@secret` field used in an `exists` or plain comparison query, UI kind not present in the active component map.

4. UI IR, emitters and component maps

Both targets are generated from one target-neutral UI IR, so supporting two frameworks costs two thin emitters rather than two compilers. All target-independent logic lives in the IR stage; an emitter only prints.

UI IR (records in `dv-compiler`): `View`, `Form`, `Field(name, kind, label, validators)`, `Action(name, signedContext)`, `Text`, `Container`, `RawTemplate(target, source)`. Validation rules, action wiring, labels and field order are resolved here, before any emitter runs.

Emitter interface

```
public interface Emitter {
    String targetId(); // "react" | "lit"
    EmittedModule emit(View view, ComponentMap map);
    ShellAssets shell(AppModel app); // index.html, router module,
import map entries
    Set<String> vendorEntryPoints(ComponentMap map, AppModel app);
}
```

Component map: a data file, not code. It maps an abstract kind to a concrete tag or component, its import specifier, and prop and event names. v1 ships two maps per target:

Target	Plain map (no library)	Library map
React	native HTML elements	MUI
Lit	native HTML elements	Carbon web components

The plain maps are the test baseline; library maps prove the mechanism. Adding a design system later means adding one map file.

Target selection: one target per app, set in `dv.config` (`target: react` or `target: lit`, plus `componentMap`). No mixing of targets inside one app in v1. The React target may still use web components through its map, since React 19 supports custom elements.

Emitted module rules

- React: function component, hooks for field state, JSX source passed through esbuild's JSX transform. Router: `react-router`.
- Lit: `LitElement` class with reactive properties, served as plain ESM with no transform. Router: `@lit-labs/router`.
- Both: the module calls the shared `dv-client` runtime (about 100 lines, hand-written once per target) for action calls, error display and client-side validation. Emitters never inline fetch logic.
- Bare import specifiers only (`react`, `lit`, `@mui/material/Button`); resolution is the import map's job (section 7).
- Never deliver templates as HTML strings for `unsafeHTML` or `dangerouslySetInnerHTML`.

Conformance test: the same `.dv` fixtures run against both emitters in a headless browser (Playwright). Each fixture asserts the same DOM-level behaviour: fields present, client validation fires, submit calls the action, server error renders.

5. LSP server and IntelliJ

The LSP server is a thin LSP4J wrapper over `dv-compiler`, launched as `java -jar dv.jar lsp` over stdio, and wired into IntelliJ through the LSP4IJ plugin with no custom plugin code. It is built second, right after the compiler core, so every later `.dv` test file is written with editor feedback.

Feature	v1 behaviour
Diagnostics	Published on open and on every change, debounced 150 ms, straight from <code>CompilationResult</code>
Completion	Top-level annotations, validator names and arguments, field kinds, UI kinds from the active component map, domain, field and query names
Hover	Declaration summary for domains, fields, queries, UI kinds
Go to definition	Domain, field and query references, across files
Document symbols	Domains, fields, database blocks, queries, components
Semantic tokens	Annotations, declarations, references, strings, <code>@JS</code> body as one token type

Requirements

- Full-document sync. Workspace scan of `**/*.dv` on initialize to build the symbol table.
- The server reads `dv.config` so completion offers only UI kinds the active component map supports.
- Log to stderr or a file only. Stdout carries the protocol; one stray `println` breaks the session.
- Ship in the repo: a TextMate grammar for `.dv` as a highlighting fallback, and an LSP4IJ server definition (command, `*.dv` mapping) that can be imported instead of typed by hand.
- The dev server and the LSP server are separate processes from the same jar. They share code, not state.

Test: LSP4J-based integration tests that open a document, send a change and assert on published diagnostics, completion items and definition locations. No IDE needed in CI.

6. Vert.x 5 BFF runtime

The runtime holds zero UI state per user: it serves compiled modules, verifies signed actions, runs them, and returns JSON. Any instance can serve any request.

Route	Purpose
GET /_dv/components/:name.:hash.js	Emitted component module, from the in-memory compile cache
GET /_dv/vendor/*	esbuild output: shared chunks and entry modules
POST /_dv/actions	Signed stateless action RPC
GET /_dv/dev/events	Dev only: SSE reload and error overlay feed
GET /*	SPA shell <code>index.html</code> with generated import map

Signed actions

- When a view is emitted for a session, each action gets a token: HMAC-SHA256 over action name, bound context values (for example `userId: 482`), session id and expiry.
- The client posts `{token, input}`. The server verifies signature and expiry, checks the token's session id against the caller's session, then runs an authorization check for that user and action. A valid signature proves the server issued the action; it does not prove this user may run it.
- Field input is validated server-side with the same rules the client used. Client validation is a convenience only.
- Responses: `{ok, data}` or `{ok: false, fieldErrors, message}`. CSRF protection on every POST.
- Signing key comes from configuration, never from source, and supports rotation (accept current and previous key).

Sessions: `vertx-web` session handler with the Redis session store; HTTP-only, Secure, SameSite=Lax cookie holding only the session id. Session content is identity and roles. Form drafts stay in the browser.

Threading

- Event loop: routing, static and cached module serving, Redis, PostgreSQL. All I/O uses the reactive clients.
- Worker pool via `executeBlocking` or virtual threads: compilation, esbuild invocation, polyglot execution. Nothing CPU-bound or process-spawning runs on an event loop.
- Vert.x 5 API only: `Future`-returning methods, `VerticleBase`, no callback overloads.

Compile cache: keyed by content hash of the `.dv` source plus target plus component map. In production mode everything is compiled at startup and the cache is read-only.

7. esbuild bundling

The runtime drives esbuild itself, so the developer never opens a second terminal. The hard requirement is a single shared copy of the framework runtime (`react` and `react-dom`, or `lit`) across every module on the page.

Vendor build: one esbuild invocation, not one per bundle

1. Collect entry points from `Emitter.vendorEntryPoints`: the framework, the router, `dv-client`, and one entry per library component the component map references.
2. Write synthetic entry files under `.dv/cache/entries/`.
3. Run esbuild once with `--bundle --format=esm --splitting --target=es2022 --outdir=.dv/cache/vendor --entry-names=[name]-[hash] --chunk-names=chunk-[hash] --metafile=meta.json`.
4. Read `meta.json` and generate the import map: each bare specifier maps to its hashed output file. The shell `index.html` embeds this map.
5. Emitted component modules keep bare imports and resolve through the map, so they share chunks with the vendor code.

Process rules

- Invoke the esbuild binary directly from `node_modules/@esbuild/<platform>/bin/esbuild` (fall back to `node_modules/.bin/esbuild`). Never go through `npx`.
- React component modules: esbuild transform with `--loader=jsx --jsx=automatic`, imports left external, result cached by content hash. Lit modules are served untransformed.
- Rebuild the vendor set only when the entry-point set or `package-lock.json` hash changes.
- Hashed filenames get `Cache-Control: public, max-age=31536000, immutable`. `index.html` gets `no-cache`.
- esbuild failures surface as DV diagnostics in the console and the browser overlay, with esbuild's message attached.

npm auto-install: on `dv dev` startup, generate or update `package.json` from `dv.config` (target, component map, esbuild). If `node_modules` is missing or the lockfile hash changed, run `npm install --no-audit --no-fund` on a worker and block the first request until it finishes. Node and npm are assumed installed; print a clear message if they are not.

dv build: same pipeline with `--minify`, all components precompiled, output to `target/dv-dist/` for the runtime to serve in production with no Node present.

8. Domain and database

A `@Domain` is the single source of truth for a type: it generates the client validator, the server validator and the table mapping, so the three cannot drift.

Validators

- Built-ins for v1: `notEmpty`, `size(min, max)`, `email`, `pattern(regex)`, `unique`, `min`, `max`.
- Each built-in has one definition that yields a JS implementation (shipped in `dv-client`) and a Java implementation. A shared JSON test-vector file runs against both in CI.
- `unique` is server-only: it compiles to an existence query and a unique index, and the client skips it.
- Custom validators reference a `@JS` function by name. They run in the browser as-is and on the server through GraalJS (section 9).

Database

- PostgreSQL through the Vert.x reactive PG client. Prepared statements only; no string-concatenated SQL anywhere in generated code.
- `crud` generates create, read by id, update, delete and a keyset-paginated list.
- `query name(args): exists | one | list` generates an equality query over the named fields.
- `query name(identifier, secretField): verify` loads the row by identifier and verifies the secret against the stored hash. It returns the entity without secret fields, or a generic failure.
- Schema: `dv dev` generates `CREATE TABLE IF NOT EXISTS` and additive columns. Destructive changes are reported as diagnostics, never applied. Production uses exported SQL migrations.

`@secret` fields

- Hashed with Argon2id on write. Never selected into responses, never logged, never placed in action tokens or client state.
- Using a `@secret` field in an `exists`, `one` or `list` query is a compile error. The transcript's `existsByUsernameAndPassword` pattern must be impossible to express.
- Login failures return one generic message and take constant time regardless of whether the user exists.

9. Dev loop and polyglot escape hatch

Dev loop (`dv dev`): one command starts everything; saving a `.dv` file updates the browser in well under a second.

- Watch `**/*.dv` and `dv.config` with the JDK `WatchService`, debounced 100 ms.
- On change: recompile the workspace, invalidate affected cache entries, push an event on `/_dv/dev/events`.
- Success sends `reload`; the client does a full page reload. Hot module replacement is out of scope.
- Failure sends `error` with the diagnostics; `dv-client` renders an overlay showing file, line and message. The same diagnostics print to the console in `file:line:col` form so IntelliJ makes them clickable.
- The dev SSE route and the overlay code are not present in `dv build` output.

Polyglot escape hatch (built last; nothing earlier depends on it)

- `@JS` functions serve two uses: emitted verbatim into the client module, and callable server-side from validators and action handlers.
- Server-side execution uses the GraalVM polyglot API from plain Java: one shared `Engine`, and a **new Context per invocation**, closed afterwards. The shared engine keeps parsed and compiled code warm, so context creation stays cheap.
- No context pooling. Clearing bindings does not reset JavaScript global state, so a pooled context can carry one user's data into another user's request.
- Context permissions: no host class lookup, no IO, no threads, no native access. Expose an explicit allow-list of host objects through `HostAccess.EXPLICIT`. Set a statement limit or timeout per invocation.
- Java service calls from DV: `onSubmit: java("com.acme.UserService.promote")` resolves at compile time to a public static or registered-bean method; a missing target is a diagnostic. This path is plain Java reflection at startup and needs no polyglot context.
- Run on a GraalVM JDK so GraalJS gets the optimizing compiler. On another JDK it runs in interpreter mode; log that once at startup rather than suppressing the warning.

10. Do not copy from the source transcript

The attached AI transcript is background for the idea only; where it conflicts with this brief, the brief wins. These specific items from it are wrong and must not appear in the code.

Transcript item	Problem	Do instead
Server returns template HTML, client renders with <code>unsafeHTML</code>	Bindings inside the string are inert; XSS hole	Emit ES modules (section 4)
<code>POST /api/actions/:actionName</code> with plain JSON	Open RPC; signing was dropped	Signed tokens plus authorization (section 6)
<code>PolyglotContextPool</code> with binding cleanup	JS globals leak between users	New context per invocation (section 9)
<code>allowAllAccess(true)</code>	Guest code gets the whole JVM	Explicit host access allow-list
<code>Context.newBuilder("viewd1", "js", "python")</code>	Python is not a dependency; fails at startup	JS only in v1
Custom <code>TruffleLanguage</code> , Truffle nodes, DSL processor	Declarative language gains nothing; large cost	Plain compiler (section 3)
<code>InvokePolyglotActionNode</code> sample	One <code>@NodeChild</code> , three-argument specialization; does not compile	Not needed
<code>Context.getCurrent()</code> + <code>eval</code> inside a node	Wrong API level	Not needed
- <code>Apolyglot.engine.WarnInterpreterOnly=false</code> compiler arg	Not a javac option	Run on GraalVM JDK; log once otherwise
<code>npx esbuild</code> with stdin entry	Slow start; stdin cannot resolve <code>node_modules</code> without <code>--resolve-dir</code>	Direct binary, entry files on disk (section 7)
One esbuild bundle per feature	Duplicate framework copies, double element registration	One build with <code>-splitting</code> plus import map

Transcript item	Problem	Do instead
<code>immutable</code> cache header on unhashed names	Stale code forever	Content-hashed filenames
<code>existsByUsernameAndPassword</code>	Plaintext password comparison	<code>@secret</code> plus <code>verify</code> query (section 8)
Javalin, Vert.x 4.5 callbacks, Java 21, GraalVM 24.1	Wrong stack and versions	Section 2 stack
<code>Runtime.availableProcessors() * 2</code> sizing, <code>executeBlocking</code> for everything	Arbitrary	Reactive clients for I/O; workers only for CPU and processes

11. Milestones and acceptance

Eight milestones, each mergeable and demonstrable on its own; M1 to M5 produce the first end-to-end demo. Build in this order and do not start a milestone before the previous one passes its check.

#	Milestone	Passes when
M1	Parent POM, modules, grammar, AST, symbol table, diagnostics	Example files parse to the model; 20 broken fixtures each give the expected diagnostic with correct range; no fixture throws
M2	<code>dv-lsp</code> plus LSP4IJ definition and TextMate grammar	In IntelliJ: errors underline while typing, completion offers validators and domain names, go-to-definition jumps across files; LSP4J integration tests green
M3	Vert.x 5 skeleton: shell, sessions on Redis, signed action endpoint, CSRF	Tampered, expired and cross-session tokens are each rejected in tests; a stub action round-trips
M4	UI IR, <code>Emitter</code> interface, React and Lit emitters with plain maps, <code>dv-client</code> for both	<code>UserLogin</code> renders and submits under both targets; Playwright conformance suite green for both
M5	esbuild service, splitting, import map, hashed output, npm auto-install, MUI and Carbon maps	Fresh clone with no <code>node_modules</code> : <code>dv dev</code> serves the login page; page loads exactly one copy of <code>react</code> or <code>lit</code>

#	Milestone	Passes when
M6	Validators on both sides, PostgreSQL CRUD, <code>verify</code> query, <code>@secret</code>	Shared validator vectors pass in JS and Java; login works against Postgres; secret misuse fixtures fail compilation
M7	Dev loop: watcher, SSE reload, overlay; <code>dv build</code>	Save-to-reload under 1 s on the example app; syntax error shows the overlay and clears on fix; <code>dv build</code> output runs with Node absent
M8	<code>@JS</code> server-side execution, Java service calls	Custom JS validator runs on both sides with the same result; a test proves no global state survives between two invocations

Working rules

- Every milestone lands with tests; CI runs unit, LSP integration and Playwright suites.
- Generated code is snapshot-tested: emitter output for each fixture is committed and diffs are reviewed.
- Anything unclear in the language design: propose the smallest option in a short note and continue. Do not extend the language beyond section 3 without sign-off.